

Build: TruckTrack — Truck Journey & Transport Control System

1. Project Objective

Build a professional internal transport-management system for a company that currently operates primarily with one truck travelling between:

Noida Headquarters → Delhi → Noida Headquarters

The director's primary problem is not simply knowing whether the truck is present.

The director wants **operational control and reliable records** without having to continuously call or question the driver.

The system must automatically and reliably answer:

- Which truck is being used?
- Which driver is operating it?
- When did the truck leave Noida?
- When did it reach Delhi?
- When did it leave Delhi?
- When did it return to Noida?
- How long did each journey take?
- What was the expected travel time?
- Was there a delay?
- How large was the delay?
- What was the reason for the delay?
- Did the driver actually perform the required punch/event?
- Are there missing, suspicious, or manually modified records?
- What is the historical travel-time pattern?
- Does the director need to take any action?

The system should reduce the director's operational headache while increasing visibility and accountability.

Do NOT build an unnecessarily large fleet-management SaaS.

Build a **focused, professional, modular transport-control system** that solves the current business problem properly and can expand later.

2. Core Product Philosophy

Follow these principles throughout development:

Principle 1 — Automation over manual reporting

The driver should not have to repeatedly send WhatsApp messages or call the director.

The system should automatically record important events wherever technically possible.

Principle 2 — Evidence over trust

The system should not assume that every driver-reported event is automatically accurate.

Create an auditable event history.

For example:

Truck identified
↓
Driver authenticated
↓
Trip started
↓
Timestamp recorded
↓
Location/event evidence recorded if available
↓
Destination arrival recorded
↓
Trip completed

The goal is not to treat the driver as untrustworthy.

The goal is to make the system reliable enough that the director does not need to personally verify every small event.

Principle 3 — Exception-based management

The director should NOT need to monitor the truck every minute.

The system should tell the director when something requires attention.

Normal operation:

No notification required.

Abnormal operation:

Surface the issue.

Examples:

- Truck has not started within the expected window.
- Truck is significantly delayed.
- Arrival punch is missing.
- Return trip has not started.
- Trip remains open unusually long.
- Delay reason has been entered.
- A record has been manually modified.
- Truck/driver assignment is inconsistent.

Principle 4 — Keep the first version practical

Current reality:

- One primary truck.
- One main route.
- Small business.
- No expectation of rapid fleet expansion.

Therefore, do not build unnecessary enterprise complexity.

However, the database and API should not be hard-coded around exactly one truck.

3. Recommended Technology

Backend:

- Python
- FastAPI
- REST API
- Pydantic
- SQLAlchemy

Database:

- PostgreSQL preferred
- SQLite may be used for local development if appropriate, but production architecture should support PostgreSQL.

Frontend/mobile:

The system must be structured so Android and iOS can communicate with the Python REST API.

If the project environment already has a chosen mobile framework, use it.

Do not introduce unnecessary technologies without a reason.

Dashboard:

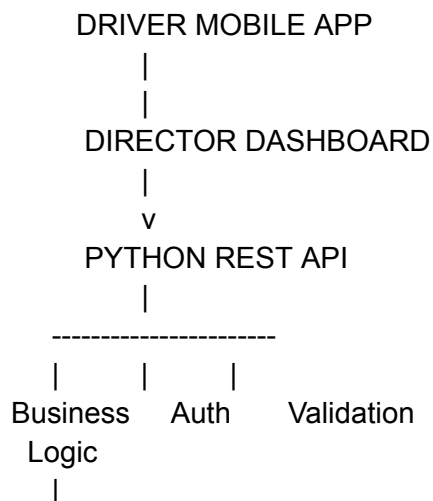
A responsive web dashboard for the director/admin.

Authentication:

- Secure login
- Role-based access
- Token/session-based authentication
- Password hashing

4. High-Level Architecture

Use this architecture:



v
SQL DATABASE

The mobile application must NEVER directly access the SQL database.

All communication must happen through the API.

5. Main User Roles

Start with only the necessary roles.

Driver

Can:

- Login
- See assigned truck
- See today's assigned trip
- Start/punch-out journey
- Punch-in/confirm arrival
- Start return journey
- Punch-in/confirm return
- Report delay
- Select delay reason
- Add remarks
- View own trip history

Director

Can:

- View dashboard
- View current trip
- View truck status
- View travel history
- View delays
- View delay reasons
- View driver/truck information
- View reports

- Filter records
- Investigate exceptions

Admin

Can:

- Manage users
- Manage drivers
- Manage trucks
- Manage routes
- Configure expected travel times
- Manage delay reasons
- Correct records when necessary
- View audit history

Keep role permissions clearly separated.

6. Truck Recognition / Identification

The system should support a concept similar to the previously developed face-recognition attendance system.

The important difference is:

Face recognition identifies a person.

This system should identify/verify the truck.

Design the system so that truck identification can eventually support:

- QR code
- Unique truck ID
- Registration-number recognition
- Camera/image-based truck recognition
- NFC/RFID if required later

For V1, implement the most reliable and practical identification method available in the project environment.

Do NOT claim that camera-based recognition is reliable unless it has actually been implemented and tested.

The system should produce:

Truck identified:
UP16AB1234

Assigned driver:
Driver Name

Current assignment:
Noida → Delhi

The truck identity must be stored against the trip.

7. Journey Model

A complete daily operation is:

```
NOIDA
|
| Punch Out / Departure
v
DELHI
|
| Punch In / Arrival
|
| Work / Loading / Activity
|
| Punch Out / Departure
v
NOIDA
|
| Punch In / Return
v
TRIP COMPLETED
```

The system must treat this as a sequence of verified events.

Do NOT simply store one "start_time" and one "end_time."

Create an event model.

Example:

TRIP-001

09:00

NOIDA_DEPARTURE

10:15

DELHI_ARRIVAL

11:00

DELHI_DEPARTURE

12:20

NOIDA_ARRIVAL

8. Trip Database Design

Create appropriate relational tables.

At minimum:

users

id

name

email/username

password_hash

role

is_active

created_at

updated_at

drivers

id

user_id

employee_id

name

phone

status
created_at
updated_at

trucks

id
registration_number
truck_code
truck_type
capacity
status
assigned_driver_id
created_at
updated_at

routes

id
origin
destination
expected_duration_minutes
allowed_delay_minutes
is_active
created_at
updated_at

trips

id
trip_number
truck_id
driver_id
route_id
trip_date
status
created_at
updated_at

trip_events

id
trip_id
event_type
event_time
latitude
longitude
verification_method
device_id
created_by
created_at

Possible event types:

NOIDA_DEPARTURE
DELHI_ARRIVAL
DELHI_DEPARTURE
NOIDA_ARRIVAL

The event model should be extensible.

delays

id
trip_id
route_segment
delay_minutes
reason_id
remarks
reported_by
created_at

delay_reasons

id
name
description
is_active

Examples:

Traffic
Vehicle Breakdown
Loading Delay
Unloading Delay
Road Closure
Accident
Weather
Driver Issue
Fuel Issue
Other

audit_logs

id
user_id
action
entity_type
entity_id
old_value
new_value
timestamp
ip/device information where appropriate

The audit system is important because management needs confidence in the records.

9. Travel-Time Calculation

The backend must calculate travel time automatically.

Example:

Noida departure:
09:00

Delhi arrival:
10:15

Actual travel time:
75 minutes

If expected travel time is:

60 minutes

Then:

delay = 75 - 60

delay = 15 minutes

Use configurable thresholds.

Do NOT hard-code a universal assumption that anything above 60 minutes is delayed.

The route configuration should determine:

Expected duration

Allowed tolerance

Example:

Expected: 60 minutes

Tolerance: 10 minutes

Actual: 65 minutes

Status:

ON_TIME

But:

Actual: 78 minutes

Status:

DELAYED

Delay:

18 minutes

10. Delay Management

When a trip becomes delayed, the system should make the reason easy to record.

Driver sees:

Trip delayed.

Select reason:

- [Traffic]
- [Vehicle Breakdown]
- [Loading/Unloading]
- [Accident]
- [Road Closure]
- [Weather]
- [Driver Issue]
- [Other]

Then:

Additional remarks:

Do not make drivers fill long forms.

The system should capture the minimum information necessary.

11. Reliability / Anti-Manipulation Design

This is an important management requirement.

The system should distinguish between:

Automatically generated event

Example:

Truck identification

Timestamp
GPS/location if available
Device information

Driver-confirmed event

Example:

Driver pressed "Arrived"

Admin correction

Example:

Original:
10:15

Changed:
10:25

Changed by:
Admin

Reason:
Driver forgot to punch

Never silently overwrite historical data.

If a record must be corrected:

Original value
+
Corrected value
+
Who changed it
+
When
+
Why

must remain auditable.

This gives the director confidence in the system.

12. Dashboard Philosophy

The dashboard is NOT supposed to display every database field.

The director wants answers, not raw data.

The dashboard should have:

A. Current Operation

Today's Trip

Truck:

UP16AB1234

Driver:

Driver Name

Current Stage:

DELHI → NOIDA

Status:

ON TIME

Started:

11:00 AM

Expected Arrival:

12:00 PM

Current Delay:

0 min

13. Today's Journey Timeline

Show a visual timeline:

09:00

- Noida Departure

|

|

10:15

- Delhi Arrival

|

|

11:00

- Delhi Departure

|

|

12:20

- Noida Arrival

Each event should show:

- Time
 - Location
 - Event
 - Verification method
-

14. Director KPI Cards

Keep these concise.

TODAY

Trip Status	ON TIME
Current Stage	Delhi → Noida
Travel Time	1h 20m
Total Delay	20m

For historical view:

Average Noida → Delhi
Average Delhi → Noida
On-Time Percentage
Average Delay
Total Trips

15. Exception Center

This should be one of the most important parts.

Example:

ATTENTION REQUIRED

⚠ Truck delayed by 32 minutes

⚠ Delhi arrival punch missing

⚠ Return journey not started

⚠ Trip open for unusually long duration

Clicking an exception should take the director directly to the relevant trip.

This prevents the director from becoming a full-time monitor.

16. Historical Analytics

Provide:

Today

This Week

This Month

Custom Range

Show:

Travel time

Noida → Delhi

Average

Minimum

Maximum

Return time

Delhi → Noida

Average

Minimum

Maximum

Delay

Total delays

Average delay

Longest delay

Most common reason

Do not add complicated analytics unless they provide a useful business decision.

17. Delay Pattern Analysis

The system should eventually answer:

What causes most delays?

Example:

Traffic	52%
Loading	21%
Vehicle Issue	15%
Other	12%

And:

Which direction has more delays?

Noida → Delhi	35%
Delhi → Noida	65%

This turns the project from a record-keeping system into a management tool.

18. Driver Accountability Without Micromanagement

The system should provide management visibility without forcing the director to manually supervise the driver.

Track:

- Trips completed
- On-time percentage
- Missing punches
- Delay reports
- Delay frequency
- Manually corrected events
- Trip completion consistency

Do not create a system whose purpose is simply to punish drivers.

The objective is:

Reliable operations + accountability + less manual supervision.

19. Reports

Allow the director/admin to generate:

Daily report

Date
Truck
Driver
Departure
Arrival
Travel time
Expected time
Delay

Reason
Status

Weekly report

Include:

- Number of trips
- Average travel time
- Average delay
- On-time percentage
- Delay reasons

Monthly report

Include trends and comparisons.

Reports should be exportable if practical.

20. Notifications

Do NOT notify the director for every normal event.

Use meaningful alerts.

Examples:

Trip started

may be visible on the dashboard without generating a disruptive notification.

But:

Truck is 30+ minutes beyond expected arrival

should generate an alert.

Make notification thresholds configurable.

21. Future-Proofing

Design the architecture so these can be added later without rebuilding the system:

GPS tracking

Live location
ETA
Route deviation
Stopped time

Multiple trucks

The current database should support:

Truck A
Truck B
Truck C

even if only one is currently active.

Multiple routes

Noida → Delhi
Delhi → Noida
Noida → Gurgaon
etc.

Maintenance

Service history
Repair records
Service reminders

Fuel

Fuel quantity
Cost
Mileage
Fuel efficiency

Vehicle documents

RC
Insurance
PUC
Permit
Expiry alerts

Only architect for these future capabilities.

Do NOT implement them in V1 unless required.

22. Security Requirements

Implement:

- Password hashing
- Authentication
- Authorization
- Input validation
- SQL injection protection
- Secure API design
- Role-based permissions
- Audit logging
- Appropriate error handling
- No passwords/secrets hard-coded in source code
- Environment variables for secrets

Never expose database credentials through the frontend.

23. API Design

Create clean REST endpoints.

Examples:

POST /auth/login

```
GET /trucks
GET /trucks/{id}

GET /drivers
GET /drivers/{id}

GET /routes
POST /routes

POST /trips
GET /trips
GET /trips/{id}

POST /trips/{id}/events

POST /trips/{id}/delay

GET /dashboard/today
GET /dashboard/analytics

GET /reports/daily
GET /reports/monthly
```

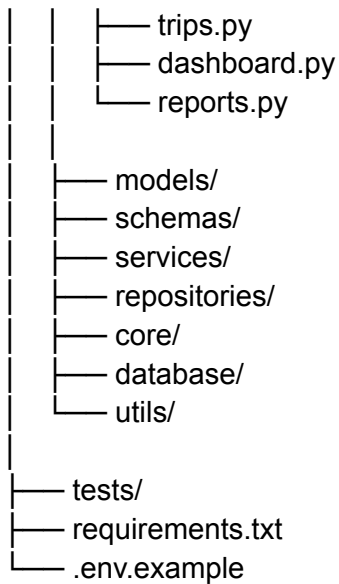
Keep business logic out of route handlers where possible.

Use a clean service layer.

24. Backend Project Structure

Use a maintainable structure similar to:

```
backend/
|
|-- app/
|   |-- main.py
|   |-- api/
|       |-- auth.py
|       |-- trucks.py
|       |-- drivers.py
```



Keep the architecture understandable for a junior developer who will maintain it.

25. Development Process

Do NOT generate the entire application blindly in one step.

Build it in stages.

STEP 1 — Requirements

Create:

requirements.md

Clearly document:

- Business problem
- Users
- Main workflow
- Functional requirements
- Non-functional requirements
- Current scope
- Future scope

- Assumptions

STOP and verify the requirements before coding.

STEP 2 — System Design

Create:

architecture.md

Include:

- System architecture
- Component responsibilities
- Data flow
- Authentication flow
- Trip lifecycle
- Event lifecycle

Also create a simple architecture diagram.

STEP 3 — Database Design

Create the ERD.

Define:

- Tables
- Primary keys
- Foreign keys
- Relationships
- Constraints
- Indexes

Pay particular attention to:

Trip

TripEvent

Truck

Driver

Route
Delay
AuditLog

The database must support more than one truck even though the current business uses one.

STEP 4 — Backend Foundation

Implement:

- FastAPI
- Database connection
- SQLAlchemy
- Configuration
- Environment variables
- Error handling
- Logging
- Health endpoint

Test the database connection.

STEP 5 — Authentication

Implement:

- Login
- Password hashing
- Authentication
- Roles
- Authorization

Test permissions.

A driver must not be able to access admin functions.

STEP 6 — Truck & Driver Management

Implement CRUD operations.

Admin can:

- Add truck
 - Edit truck
 - Activate/deactivate truck
 - Add driver
 - Assign driver
 - View truck details
-

STEP 7 — Trip Lifecycle

Implement the central workflow:

Create Trip
↓
Assign Truck
↓
Assign Driver
↓
Noida Departure
↓
Delhi Arrival
↓
Delhi Departure
↓
Noida Arrival
↓
Trip Completed

Prevent invalid sequences.

For example:

A driver cannot record:

NOIDA_ARRIVAL

before:

DELHI_DEPARTURE

unless an authorized correction is made.

STEP 8 — Time & Delay Engine

Implement:

expected_duration
actual_duration
allowed_tolerance
delay_minutes
status

Test many scenarios.

Examples:

- Exactly on time
 - Within tolerance
 - Delayed
 - Missing event
 - Duplicate event
 - Invalid event order
-

STEP 9 — Delay Reporting

Implement:

- Delay reason
 - Remarks
 - Timestamp
 - Reporter
 - Trip association
-

STEP 10 — Dashboard API

Build APIs that return management-level information rather than raw database records.

Example:

GET /dashboard/today

should return:

current_trip
current_stage
truck
driver
travel_time
expected_time
delay
status
exceptions

STEP 11 — Director Dashboard

Build the UI around:

1. Current status
2. Today's timeline
3. Expected vs actual time
4. Exceptions
5. Historical performance
6. Delay analysis

Keep it clean and professional.

The director should understand the current situation in seconds.

STEP 12 — Driver Mobile Interface

Build a very simple workflow.

The driver should not navigate through complicated menus.

Home:

Assigned Truck
Current Trip
Current Stage

[START / PUNCH]

Then the appropriate next action should appear.

Avoid allowing the driver to manually enter times unless absolutely necessary.

The application should use the device's current timestamp for normal events.

STEP 13 — Verification / Truck Recognition

Integrate the chosen truck-identification method.

The system should associate:

Truck Identity
+
Driver Identity
+
Trip
+
Timestamp

Test incorrect truck identification and unauthorized combinations.

STEP 14 — Audit System

Implement audit logs for important changes.

Never silently overwrite important operational records.

STEP 15 — Testing

Create tests for:

Authentication

- Correct login
- Incorrect password
- Unauthorized access

Trips

- Correct event sequence
- Incorrect sequence
- Duplicate event
- Missing event

Delay

- On-time
- Delayed
- Tolerance boundary

Permissions

- Driver
- Director
- Admin

Data integrity

- Invalid truck
- Invalid driver
- Invalid trip
- Deleted/deactivated entities

STEP 16 — Demo Data

Create realistic seed data.

At minimum:

1 truck

1 driver

2 routes
30+ historical trips
Multiple delay reasons
Some delayed trips
Some on-time trips

This will make the dashboard meaningful during demonstration.

26. UX Requirement

The system must be designed for two completely different users.

Driver

Needs:

Simplicity

Minimum actions.

Large clear buttons.

Very little typing.

Director

Needs:

Visibility

Dashboard.

Exceptions.

Trends.

Reports.

No unnecessary operational details unless they click into a trip.

This distinction is critical.

27. What NOT to Do

Do not:

- Hard-code the single truck into the application.
 - Hard-code Noida/Delhi everywhere.
 - Let mobile clients access SQL directly.
 - Store passwords as plain text.
 - Allow arbitrary modification of timestamps.
 - Build dozens of unnecessary features.
 - Build GPS before establishing whether it is actually required.
 - Build AI just to make the project sound advanced.
 - Create microservices for a small internal system.
 - Overcomplicate the UI.
 - Make the driver fill long forms.
 - Notify the director about every minor event.
 - Generate code without testing each module.
-

28. Definition of Done

The project is successful when this real-world scenario works:

Morning

Driver logs in.

System identifies/validates the assigned truck.

Driver starts the journey.

System records:

Truck

Driver

Trip

Noida departure

Timestamp

Verification information

Delhi

Driver records arrival.

System calculates:

Expected time

Actual time

Delay

Status

If delayed, the driver records the reason.

Return

Driver starts the return journey.

System records Delhi departure.

Driver arrives at Noida.

System records final arrival.

Trip becomes:

COMPLETED

Director

Director opens dashboard.

Immediately sees:

Today's Trip

Current Status

Travel Time

Expected Time

Delay

Delay Reason

Journey Timeline

Director can then look at historical data and understand whether delays are isolated or recurring.

The director should not need to call the driver simply to answer basic operational questions.

29. Product Direction

The final system should feel like:

"A digital control layer over the company's transport operation."

Not:

"An attendance form for a truck."

The current business is small.

The system should therefore be:

Simple in operation + structured in architecture + reliable in records + useful for management.

Build V1 around the actual business problem.

Make the architecture extensible.

Do not build hypothetical features merely because they are technically possible.

30. Final Instruction to start

Before writing application code:

1. Analyze this specification.
2. Identify ambiguities and assumptions.
3. Produce the proposed system architecture.
4. Produce the database ERD/schema.
5. Produce the API specification.
6. Produce the user flows.
7. Produce the development plan.

8. Explain important technical decisions.
9. Identify anything that should be clarified with the company.
10. Only after the design is internally consistent, begin implementation.

Implement the project incrementally.

After each major module:

- Run tests.
- Check errors.
- Verify database integrity.
- Verify API responses.
- Verify authorization.
- Verify the actual user workflow.

Do not replace working components unnecessarily.

Prioritize:

Correctness → Reliability → Security → Usability → Maintainability → Scalability → Advanced features.